Ticket Triage Agent: End-to-End Project Flow Guide

Audience: Team members, technical leads, project leads, and demo reviewers.

Purpose: Explain how the project works step by step, from ticket submission to final triage response, admin analytics, memory, guardrails, and auditability.

1. Executive Summary

The Ticket Triage Agent is an AI-assisted support workflow that receives a user ticket, analyzes the issue, finds a relevant solution from an internal knowledge base when possible, falls back to general LLM guidance when the knowledge base is weak, classifies the ticket, detects priority, routes it to the right support team, applies high-risk guardrails, and stores a complete audit trail for admin review.

The application has two main surfaces:

- User mode: submit a ticket and receive a simplified triage result.
- Admin mode: view richer triage output, audit trails, RAG usage, web fallback usage, guardrail warnings, analytics, and saved ticket history.

The backend exposes FastAPI endpoints. The frontend is a React application using Vite. Ticket logs are stored locally in JSON, while the retrieval pipeline uses embeddings and a Chroma vector database built from internal ticket knowledge base data.

2. Business Problem

Support teams usually receive tickets in many forms: access issues, outages, security concerns, billing questions, software requests, hardware problems, and vague general support requests. Manual triage takes time because an agent must read the issue, understand severity, decide routing, check previous knowledge, and decide whether escalation is required.

This project automates the first pass of that workflow:

- It reduces repetitive analysis.
- It makes ticket routing more consistent.
- It surfaces suggested resolution steps.
- It tracks whether internal knowledge was used.
- It highlights high-risk issues using guardrails.
- It gives admins an audit trail for explainability.

3. High-Level Architecture

Main layers:

- Frontend: React UI for user ticket submission and admin dashboards.
- API layer: FastAPI endpoints for triage, tickets, health checks, and analytics.
- Agent orchestration: the central ticket processing flow in backend/agent.py.
- RAG layer: retrieves related internal knowledge base chunks using embeddings and ChromaDB.
- LLM layer: Groq-backed generation for suggested solutions and fallback guidance.
- Rule-based tools: classification, priority detection, routing, and guardrails.
- Memory layer: JSON ticket log storage and lookup for repeated issues.
- Analytics layer: computes counts for RAG usage, web fallback usage, and guardrail events.

Simple architecture flow:

User or Admin -> React Frontend -> FastAPI Backend -> Triage Agent -> RAG / LLM / Rules / Memory -> Response -> Frontend Display

4. Main Folder Structure

Important paths:

- frontend/src/App.jsx: main React application shell and role switching.
- frontend/src/components/TicketForm.jsx: user ticket input form.
- frontend/src/components/ResultPanel.jsx: triage result display.
- frontend/src/components/analyticsdashboard.jsx: admin analytics cards and guardrail breakdown.
- frontend/src/components/AdminTicketViewer.jsx: admin ticket history and audit viewer.
- frontend/src/api/triageApi.js: frontend API call to backend triage endpoint.
- backend/main.py: FastAPI routes.
- backend/agent.py: central orchestration logic.
- backend/models.py: request and response models.
- backend/rag/augmentation.py: RAG context and LLM answer generation.
- backend/rag/retreiver.py: vector retrieval from ChromaDB.
- backend/tools/classification.py: category detection rules.
- backend/tools/priority.py: priority detection rules.
- backend/tools/routing.py: team routing rules.
- backend/tools/guadrails.py: high-risk guardrail rules.
- backend/tools/web_search.py: fallback guidance when RAG is weak.
- backend/memory/memory_manager.py: ticket log storage and normalization.
- backend/memory/analytics_manager.py: backend analytics summary.
- backend/memory/ticket_logs.json: saved ticket records.

5. Frontend Flow

Step 1: The user selects an access mode.

The app has two roles:

- User
- Admin

The role is held in React state inside App.jsx. It controls which components are shown and what backend response shape is returned.

Step 2: User mode shows the ticket form.

The form collects:

- User name
- User email
- Department
- Subject
- Description

When the form is submitted, App.jsx calls triageTicket() from frontend/src/api/triageApi.js.

Step 3: The frontend sends the request.

The frontend POSTs the ticket payload to:

POST /triage

It also sends the current role in the x-user-role header.

Step 4: The result panel displays output.

The result panel shows:

- Ticket ID
- Category
- Priority
- Assigned team
- Suggested solution

For admins, it also shows:

- Confidence
- RAG score
- RAG used
- Web search used
- RAG sources
- Audit trail
- Guardrail status when triggered

Step 5: Admin mode shows analytics and ticket audit views.

Admin mode displays:

- RAG Used count
- Web Search Used count
- Guardrail Count
- Guardrail warning box with the latest guardrail reason
- Clickable guardrail breakdown by Production Outage, Security Incident, and Data Loss
- Ticket audit dashboard with ticket history and details

6. Backend API Flow

The backend starts in backend/main.py.

Important endpoints:

- GET /: confirms the API is running.
- GET /health: health check endpoint.
- POST /triage: runs the full triage agent.
- GET /tickets: returns saved tickets.
- GET /tickets/{ticket_id}: returns one saved ticket.
- DELETE /tickets: clears memory, admin only.
- GET /analytics: returns backend analytics summary, admin only.
- GET /analytics/tickets: returns normalized ticket logs for frontend analytics, admin only.

Role behavior:

- If x-user-role is admin, the backend returns the full TriageResponse.
- If role is user, the backend returns UserTriageResponse, which hides admin-only fields such as confidence, RAG score, sources, audit trail, and guardrail flags.

This keeps normal users focused on the outcome while admins can inspect the reasoning.

7. Request and Response Models

The input model is TicketRequest.

Fields:

- user_name
- user_email
- department
- subject
- description

The full admin response is TriageResponse.

Fields:

- ticket_id
- category
- confidence
- priority
- assigned_team
- rag_score
- rag_used
- web_search_used
- suggested_solution
- rag_sources
- audit_trail
- guardrail_applied

The user response is UserTriageResponse.

Fields:

- ticket_id
- category
- priority
- assigned_team
- suggested_solution

8. Full Ticket Lifecycle

Step 1: Ticket ID is generated.

When run_ticket_triage() starts, it creates a ticket ID in this format:

TICKET-XXXXXXXX

The ID is added to the audit trail.

Step 2: Ticket text is prepared.

The agent builds:

- issue_text: subject and description, used for analysis.
- full_ticket_text: subject, description, user name, email, and department.

Most decisions use the issue text because routing and classification should focus on the problem itself.

Step 3: Cache lookup happens first.

The agent checks existing ticket logs in reverse order. If a similar issue is found, it returns the cached ticket result and appends an audit entry saying the result came from cache.

This helps repeated issues avoid full reprocessing.

Step 4: RAG augmentation runs.

The agent calls:

augmentation_manager.build_augmented_context(issue_text)

This retrieves internal knowledge base chunks and uses them to generate suggested solution steps when the match is strong enough.

Step 5: Web fallback runs only when RAG is weak.

If RAG is not used, the agent sets web_search_used to true and calls:

web_search_fallback(issue_text)

In this project, the fallback is LLM-generated general support guidance, not a live browser search. It is called web fallback because it represents external-style general guidance after the internal knowledge base fails to match strongly.

Step 6: Classification runs.

The ticket is categorized using keyword-based rules in classification.py.

Example categories:

- Production Outage
- Security Incident
- Access Issue
- Network Issue
- Hardware Issue
- Desktop Issue
- Email Issue
- Software Request
- Bug
- Performance Issue
- Billing Issue

- Feature Request
- General Support

Step 7: Priority detection runs.

Priority is detected using keyword rules in priority.py.

Possible priorities:

- Critical
- High
- Medium
- Low

Critical examples include production outage, system down, service unavailable, complete outage, all users affected, and business critical outage.

High examples include data loss, security breach, phishing, malware, account compromise, suspicious login, and blocked work.

Step 8: Routing runs.

The agent calls:

route_ticket(category, issue_text)

Examples:

- Production Outage -> Incident Response Team
- Security Incident -> Security / IAM Team
- Data Loss Incident -> Data Support Team
- Access Issue -> Security / IAM Team
- Network Issue -> Network Operations Team
- Email Issue -> Messaging / Email Support Team
- Hardware Issue -> Desktop Support Team
- Bug with API/database/server keywords -> Backend Team
- Feature Request -> Product Team

Step 9: Guardrails are applied.

The agent calls:

apply_guardrails(category, priority, assigned_team, issue_text)

Guardrails can override category, priority, and assigned team when high-risk keywords are detected.

Step 10: Confidence is calculated.

Confidence logic:

- If RAG was used, confidence is based on the RAG score.
- If web fallback was used, confidence is 0.70.
- If neither was used, confidence is 0.50.

Step 11: Final response is created.

The response includes triage output, suggested solution, RAG metadata, audit trail, and guardrail status.

Step 12: Ticket is saved to memory.

The response is saved in:

backend/memory/ticket_logs.json

9. RAG Pipeline Explained

RAG means Retrieval Augmented Generation.

In this project, RAG works like this:

1. The incoming ticket text is converted into an embedding.
2. The embedding is compared against the ChromaDB vector collection.
3. The retriever fetches the top matching knowledge base chunks.
4. A distance score is converted into a similarity score.
5. If the best score is at least 0.50, RAG is considered strong enough.
6. The retrieved chunks are formatted as context.
7. The LLM is prompted to use only the internal knowledge base context.
8. The LLM response is converted into up to six suggested solution steps.

Important files:

- backend/rag/retreiver.py
- backend/rag/augmentation.py
- backend/rag/vector_store.py
- backend/rag/embeddings.py
- backend/llm/groq.py

Why this matters:

- It keeps answers grounded in internal support knowledge.
- It provides source tracking through rag_sources.
- It gives admins a rag_score to understand match quality.

10. Web Fallback Explained

When RAG does not find a strong internal knowledge match, the system uses fallback guidance.

Trigger condition:

- rag_used is false.

What happens:

- web_search_used becomes true.
- The backend calls web_search_fallback(issue_text).
- The fallback prompt asks for issue understanding, likely cause, steps, escalation, and a note that the answer is general guidance.
- The generated text is converted into suggested solution steps.

Project note:

The current implementation uses the Groq LLM as a general fallback assistant. It does not perform live web browsing.

11. Classification Explained

Classification is rule-based.

The system lowercases the ticket text and checks keyword groups in order.

Examples:

- Outage keywords produce Production Outage.
- Security keywords produce Security Incident.
- Login/password/access keywords produce Access Issue.
- VPN/Wi-Fi/network/connectivity keywords produce Network Issue.
- Monitor/keyboard/printer/laptop keywords produce Hardware Issue.
- API/database/server/backend/error keywords produce Bug.

If no keyword group matches, the category becomes:

General Support

Why rule-based classification is useful here:

- It is transparent.
- It is easy to explain to teams and leads.
- It is predictable during demos.
- It can be improved by adding or adjusting keyword groups.

12. Priority Explained

Priority is also rule-based.

Critical:

- production down
- system down
- application down
- service unavailable
- complete outage
- major outage
- all users affected
- multiple users affected
- business critical

High:

- data loss
- deleted data
- missing data
- database corrupted
- security breach
- phishing
- malware
- suspicious login
- account compromised
- credentials stolen
- unauthorized access
- work blocked

Medium:

- not working
- failed
- error
- delay
- slow
- timeout
- intermittent
- unable to access

Low:

- request
- information
- how to
- need help
- question
- minor
- low impact

13. Routing Explained

Routing maps the detected category and keywords to a support team.

Examples:

- Production Outage -> Incident Response Team
- Security Incident -> Security / IAM Team
- Access Issue -> Security / IAM Team
- Network Issue -> Network Operations Team
- Email Issue -> Messaging / Email Support Team
- Hardware Issue -> Desktop Support Team
- Desktop Issue -> Desktop Support Team
- Software Request -> Software Support Team
- Billing Issue -> Billing Support Team
- Feature Request -> Product Team
- General Support -> General Support Team

For Bug, the routing logic checks more details:

- API/database/server/backend/endpoint -> Backend Team
- UI/button/page/frontend/screen/form -> Frontend Team
- monitor/keyboard/mouse/printer/display/hardware -> Desktop Support Team
- otherwise -> Application Support Team

14. Guardrails Explained

Guardrails are the project’s high-risk escalation layer.

They are only triggered for:

- Security incidents
- Production outages
- Data loss incidents

When a guardrail triggers, it can override:

- category
- priority
- assigned team
- human review flag
- audit trail reason

Security guardrail:

- Category becomes Security Incident.
- Priority becomes Critical.
- Team becomes Security / IAM Team.
- Human review is forced.

Production outage guardrail:

- Category becomes Production Outage.
- Priority becomes Critical.
- Team becomes Incident Response Team.
- Human review is forced.

Data loss guardrail:

- Category becomes Data Loss Incident.
- Priority becomes High.
- Team becomes Data Support Team.
- Human review is forced.

Why guardrails matter:

- They protect high-risk issues from being under-prioritized.
- They make escalation consistent.
- They create an audit trail explanation.
- They help leads quickly see critical events in analytics.

15. Audit Trail Explained

The audit trail is a list of processing events saved with every ticket.

Typical audit entries include:

- Ticket created with ID
- Ticket text prepared
- RAG augmentation status
- RAG used
- RAG score
- Web fallback status
- Detected category
- Detected priority
- Routed team
- Guardrail reason or no-guardrail message
- Final category after guardrails
- Final priority after guardrails
- Final assigned team after guardrails
- Confidence
- Final response generated

This gives admins explainability. During a demo, the audit trail is the best place to show that the system is not a black box.

16. Memory and Repeated Issues

The memory layer stores ticket logs in:

backend/memory/ticket_logs.json

The memory manager:

- Initializes the log file.
- Saves normalized ticket logs.
- Reads all logs.
- Gets one ticket by ID.
- Clears memory for admins.
- Normalizes audit trail and guardrail flags.

Repeated issue handling:

- The agent checks old tickets before running full triage.
- If the current issue appears inside an existing issue and solution text, the existing ticket is reused.
- The audit trail notes that the result came from cache.

Benefit:

- Faster response for recurring issues.
- Demonstrates a simple memory capability.

17. Analytics Explained

Admin analytics are shown in the frontend dashboard.

Current key boxes:

- RAG Used
- Web Search Used
- Guardrail Count

Guardrail count can be opened to show:

- Production Outage count
- Security Incident count
- Data Loss count

When guardrail count is greater than zero, the analytics view also shows a red warning box with the latest guardrail reason, such as:

Guardrail applied: Production outage detected. Priority changed to Critical and ticket routed to Incident Response Team.

Purpose:

- Help leads understand how often the system used internal knowledge.
- Help teams see when fallback guidance was needed.
- Help project leads identify high-risk ticket patterns quickly.

18. Admin Ticket Viewer Explained

The admin ticket viewer allows admins to:

- Search by ticket ID.
- Filter by category.
- Filter by priority.
- Page through ticket history.
- Open a ticket.
- Inspect details.
- View suggested solution.
- View audit trail.

This view supports operational debugging and stakeholder review.

19. User vs Admin Experience

User experience:

- Submit ticket.
- See ticket ID, category, priority, assigned team, and suggested solution.
- No internal audit details are shown.

Admin experience:

- See full result details.
- See RAG usage.
- See web fallback usage.
- See RAG sources.
- See guardrail trigger status.
- See audit trail.
- See analytics.
- See all saved tickets.

This separation keeps the user interface simple while preserving transparency for admins and leads.

20. Demo Script for Teams and Leads

Demo 1: Normal ticket

Example:

Subject: VPN not connecting

Expected story:

1. User submits ticket.
2. System creates ticket ID.
3. RAG tries to find internal knowledge.
4. Classification detects Network Issue.
5. Priority becomes Medium or Low depending on wording.
6. Routing sends it to Network Operations Team.
7. Suggested solution appears.
8. Admin can inspect audit trail.

Demo 2: Weak knowledge base match

Example:

Subject: Need help with a rare application issue

Expected story:

1. RAG score is weak.
2. System marks RAG Used as No.
3. Web Search Used becomes Yes.
4. Suggested solution comes from fallback guidance.
5. Analytics Web Search Used count increases.

Demo 3: Production outage guardrail

Example:

Subject: Production system down

Description: Multiple users are affected and the service is unavailable.

Expected story:

1. Guardrail triggers.
2. Category becomes Production Outage.
3. Priority becomes Critical.
4. Team becomes Incident Response Team.
5. Analytics Guardrail Count increases.
6. Red warning box shows the latest guardrail reason.
7. Guardrail breakdown shows Production Outage count.

Demo 4: Security guardrail

Example:

Subject: Suspicious login detected

Description: User account may be compromised.

Expected story:

1. Guardrail triggers.
2. Category becomes Security Incident.
3. Priority becomes Critical.

4. Team becomes Security / IAM Team.
5. Security Incident count increases in the guardrail breakdown.

Demo 5: Data loss guardrail

Example:

Subject: Missing data after update

Description: Important records are missing from the database.

Expected story:

1. Guardrail triggers.
2. Category becomes Data Loss Incident.
3. Priority becomes High.
4. Team becomes Data Support Team.
5. Data Loss count increases in analytics.

21. Step-by-Step Talking Points

For teams:

- The frontend collects ticket details.
- The backend validates the request with Pydantic.
- The agent orchestrates all processing.
- RAG checks internal knowledge first.
- Fallback guidance runs only when internal knowledge is weak.
- Rules classify, prioritize, and route.
- Guardrails protect high-risk incidents.
- The full result is saved in memory.
- Admins can inspect every decision in the audit trail.

For leads:

- The project reduces first-level triage effort.
- The system is explainable because rules and audit trails are visible.
- Guardrails reduce risk by forcing escalation for outage, security, and data-loss signals.
- Analytics help measure knowledge-base usage and fallback dependency.
- The architecture is modular, so each part can be improved independently.

For project leads:

- The solution demonstrates end-to-end AI workflow integration.
- It combines RAG, LLM fallback, deterministic rules, role-based responses, memory, and analytics.
- It is suitable for demos because high-risk scenarios produce visible, explainable outcomes.
- Future work can add authentication, database persistence, live web search, better semantic cache matching, and automated ticket creation in tools like Jira or ServiceNow.

22. Strengths of the Current Implementation

- Clear end-to-end flow.
- Simple user/admin role split.
- Explainable rule-based classification and routing.
- RAG-first design.
- Fallback path when RAG is weak.
- Guardrails for high-risk scenarios.
- Audit trail for transparency.
- Local memory for repeated issues and analytics.
- Modular backend files.
- Frontend analytics for quick stakeholder review.

23. Known Limitations

- Role handling is header-based, not full authentication.
- Ticket logs are stored in JSON, not a production database.
- Web fallback is LLM-generated guidance, not live internet search.
- Classification and priority are keyword-based, so wording can affect results.
- Cache matching is simple substring matching, not semantic similarity.
- Guardrails only cover production outage, security incident, and data loss.
- The RAG quality depends on the vector database and knowledge base content.

24. Recommended Future Enhancements

Short-term:

- Fix encoding artifacts in frontend text.
- Add unit tests for classification, priority, routing, and guardrails.
- Improve cache matching using embeddings.
- Add more analytics filters.
- Add export options for tickets and analytics.

Medium-term:

- Replace JSON memory with SQLite or PostgreSQL.
- Add real authentication and role-based access control.
- Add live web search through an approved search API.
- Add confidence explanations.
- Add richer guardrail categories.
- Add feedback buttons for suggested solutions.

Long-term:

- Integrate with Jira, ServiceNow, Zendesk, or Freshdesk.
- Add SLA prediction.
- Add team workload-aware routing.
- Add automatic escalation notifications.
- Add dashboard trends over time.
- Add model evaluation and regression tracking.

25. Final One-Minute Explanation

This project is an AI-powered ticket triage system. A user submits a support ticket from the React frontend. The FastAPI backend receives it and the agent prepares the ticket text, checks memory for repeated issues, runs RAG against the internal knowledge base, falls back to general LLM guidance if RAG is weak, classifies the issue, detects priority, routes it to the right team, applies guardrails for high-risk outage, security, and data-loss cases, calculates confidence, saves the full audit trail, and returns a role-based response. Admins can then review analytics, guardrail warnings, ticket history, and audit trails to understand exactly how each decision was made.